Development of IoT applications based on the MicroPython platform for Industry 4.0 implementation

Gabriel Gaspar*, Peter Fabo+, Michal Kuba°, Jana Flochova*, Juraj Dudak* and Zuzana Florkova~

*Faculty of Materials Science and Technology in Trnava, Slovak University of Technology in Bratislava, Trnava, Slovak republic, gabriel.gaspar@stuba.sk, jana.flochova@stuba.sk, juraj.dudak@stuba.sk
+TNtech, s.r.o , Lucna 1014/9, Bytca, Slovak republic, fabo.peter@gmail.com
°Faculty of Electrical Engineering and Information Technology ,University of Žilina, Zilina, Slovak republic, michal.kuba@feit.uniza.sk
~Research Centre, University of Žilina, Zilina, Slovak republic, zuzana.florkova@rc.uniza.sk

***Abstract*— IoT and their industrial implementation aimed at Industry 4.0 is of a major interest among both research and development professionals. This article deals with the modifications and extensions possibilities of the standard MicroPython implementation on the STM32 platform, as well as the use in the development, implementation and programming of IoT peripherals, creation of custom libraries and custom modules at the hardware level. Discussed are the essentials associated with the development of general IoT application and the MicroPython environment, as a tool for rapid-prototyping of such applications. Portability is ensured thanks to the intensive utilization of HAL (Hardware Abstraction Layer) technology, which allows usage across the entire spectrum of devices with ported MicroPython. Modularity, scalability and extendibility of the application are demonstrated in examples, where the connection of a common LM92 temperature sensor, the control of a microcontroller pin and a DDS (Direct Digital Synthesis) generator are shown.**

Keywords— MicroPython, IoT, Industry 4.0, STM32, HAL

I. INTRODUCTION

IoT and their industrial implementation aimed at Industry 4.0 is of a major interest among both research and development professionals. New devices are developed almost on daily basis and bring new, required properties to the areas of their use, not limited just to the industrial automation, but also building automation and HVAC areas [1] and also in civil engineering [2]. Growing number of new, modern devices with enhanced properties also brings new requirements on development tools and applications. Developing a general IoT application consists of two elementary parts [3]:

- hardware control design development – input/output devices and sensing elements, HMIs and indicating elements and output devices, actuators and motors. Development of this part can be effectively performed on multiple levels - from a simple task of connecting a standard device to a standard interface (I2C, SPI,1-Wire, etc.), through creation of API for its operations, to development of own-design of a specialized sensor or output device with custom firmware. Such a firmware may contain customized implementation of communication interface, protocol implementation, data processing and unique API,
- custom application design development - from a simple single purpose application to an extensive system communicating over the Internet utilizing latest protocols and encryption methods. Since individual development of application ground-up could be extensively time consuming, a number of operating systems for the development of IoT applications have been created in the recent years. A brief comparison of such operating systems is in [4]. A typical representative of such systems is the Zephyr project [5], which is ported to extensive number of more than 200 platforms. For each and every platform it has implemented full control of basic peripherals. Build on a microkernel or nanokernel depending on the device in the means of performance, supports methods of multithreading and non-preemptive and preemptive scheduling. C and C ++ are exclusively used as programming languages.

It is a fact that the development of an IoT application requires the creation and utilization of development tools for preparation and debugging of application designed. This can consume a considerable time, depending on the project scope and complexity. Efficient way to simplify this issue is using a flexible universal tool, enabling interactive design and testing of individual parts of the IoT application. In this article we will go through the options and possibilities provided by utilization of the Python language as a development tool for IoT applications.

II. MICROPYTHON AS A TOOL FOR RAPID PROTOTYPING IOT DEVICES

Python is well known as a high-level programming language, quite a long-way from hardware. It could be also considered an excellent tool that is many times quite useful in hardware development. MicroPython [6] is a transcript of CPython reference implementation of the Python language, and is aimed for usage in microcontrollers. The implementation is already ported to several different target platforms and it is scalable and open-source. It gained its popularity in recent years in the community of developers, which is also evidenced by the fact that there are currently over 2,000 different forks on the github. These forks deal with various modifications, extensions and adaptations for multiple development and experimental boards.

The utilization of MicroPython is fairly simple, as the firmware is loaded by standard flashing software for a specific type of microcontroller into its flash memory and after it typically communicates using a terminal application on a computer using emulation of a serial interface. Main use targets of MicroPython and its forks can be sorted into following categories:

978-1-7281-5602-6/20/$31.00 ©2020 IEEE

- Education purposes - MicroPython allows using a REPL (Read – Eval – Print Loop) method to interact with the microcontroller. It makes possible accessing the microcontroller peripherals using terminal emulator without the need to program the code for initialization of peripherals and elementary communication. This way, it is possible explaining to students the basic principles of data acquisition and processing on multiple supported boards, in a simplified way using higher programming language.
- Development and testing of device and sensor designs - MicroPython provides verified, debugged and tested reference implementations of interfaces used in microcontrollers. This solves one of the developer's tasks of implementing the entire vertical structure associated with peripheral communication setup and control. Currently used integrated peripherals communicating via several serial interfaces (I2C, SPI, CAN, 1-Wire etc.) are controlled by writing and reading values, often using multiple different registers for enhanced properties of the given device, with different meaning of individual bits and interconnection of values. MicroPython provides with direct interactive access to device registers, makes it easy to verify the functionality of peripherals, develop and test appropriate hardware parts and devices, as well as algorithms for control and data acquisition from the device. The abstraction of hardware and the versatility of implementation allows using other platform than the one used originally for development, without the need for detailed knowledge of the new target platform programming.

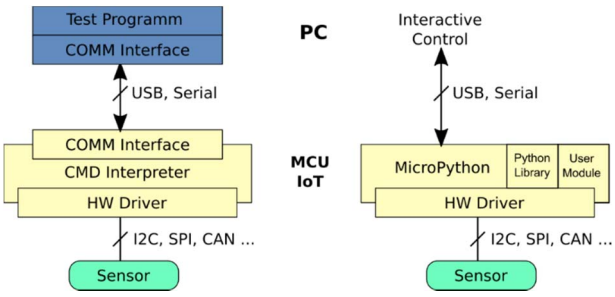

Fig. 1 Comparison of development environments

- Monitoring and configuration tool used in design of complex applications - specific applications on high performance microcontrollers and FPGA solutions often include the implementation of small, independent tools used for state monitoring and system parameters set-up. FPGAs often utilize a software implementation of a small microprocessor e.g. 8051, Z80, etc. Creating a program for such microprocessors, can be ensured state monitoring and set-up of system parameters and other configuration parameters. The one of first attempts to implement MicroPython on an FPGA [7], shows the prospect of the further development. In the field of operating systems, there are already several experimental MicroPython ports for monitoring and setting Linux kernel parameters [10]. The situation is quite simpler with microcontrollers, there is available a direct port of MicroPython as an application that can be used in the Zephyr project [8].

It is obvious that using a suitable design of a device with a microcontroller and attached various peripheral devices, we can reach a state, where it is possible to simply replace MicroPython based firmware with device firmware or vice versa, both during the development state of the application and in the production state of an IoT application. A comparison of the general development environment architecture and the alternative using MicroPython solution is shown in Fig. 1.

A. Possibilities of using STM32 platform in developing of IoT applications

STM32 microcontroller platform can be used as a platform for demonstrating possibilities of using MicroPython in the development of IoT applications. Family of STM32 microcontrollers is quite popular among developer due to its availability, price and support. It contains a wide range of microcontroller types that vary in performance, memory size, built-in peripherals and interfaces and power optimization modes. Although the STM32 platform is not fully officially supported in the MicroPython, there are available ports for some common members of this platform directly in the MicroPython source code, and the compilation of the firmware for the selected type of microcontroller is effortless thanks to the thorough documentation. Installation of MicroPython on the STM32 platform is well documented and can be done in a user-friendly way.

B. Portability of the code

Thanks to the intensive utilization of HAL (Hardware Abstraction Layer) technology, MicroPython allows for portability of the developed code not only among microcontrollers within a single family or platform, but also across the whole spectrum of supported devices from various manufacturers with ported MicroPython. In the following basic coding example, data are read from a temperature sensor and a temperature comparator LM92 [8] which is connected to the internal interface I2C bus 1. The data are read from the TEMPERATURE REGISTER as 2-byte blocks at register address number 0. The temperature sensor has 7 registers for data reading and writing the thresholds of comparators and hysteresis settings.

```
import machine
def read_LM92_TR(ic, addr):
    raw = ic.readfrom_mem(addr, 0, 2)
  # read 2 bytes from register 0
    data = (raw[0] << 8) + raw[1]
# 16 bit format conversion
    td = data >> 3       # 2'compl b15-b3 temperature
    TEMP = (-(td & 0x1000) | (td & 0xFFF))* 0.0625
    L = data & 0x01             # T_LOW  -> H, TEMP < 10 deg
    H = (data & 0x02) >> 1 # T_HIGH -> H  TEMP > 64 deg
    C = (data & 0x04) >> 2 # T_CRIT -> H  TEMP > 80 deg
    return TEMP, C, H, L
# MCU STM32L432KC
ic=machine.I2C(1)# init I2C interface, PA10-SDA, PA9-SCL
print(ic.scan())      # -> [75]  list of all devices in I2C(1)
```

print(read_LM92_TR(ic, 75)) # -> (24.5625, 0, 0, 0)

Given basic program example will work and be fully functional not only on used microcontroller and all supported microcontrollers from the STM32 family, but also on any device produced by any other manufacturer with ported MicroPython with an I2C interface. This way, if mode demanding requirements are put on the utilization of peripheral equipment, it is possible carrying out the development itself on a high-performance microcontroller and use a lower range microcontroller for the final application deployment.

C. Code modularity

MicroPython has a feature, that allows adding a new code i.e. frozen module, as a library that will become a part of the developed firmware. This prevents from repeatedly downloading already debugged code into the MicroPython environment after microcontroller resets. This feature is activated by a quite simple way and consists in saving the developed library code file (in Python language) to the directory structure at *.ports/stm32/modules* shown in Fig. 2. After this operation we need to compile the firmware and upload it to the target microcontroller. The library is then fully available using the commonly used *import* command.

```
micropython
 |
 +--ports
 |   |
 |   +--stm32
...  |   |
     ...  +--my_modules      <--- created directory
          |   |
          ...  +--demo.py      <--- module code
```

Fig. 2 Directory structure

When compiling, we need to define the variable *FROZEN_DIR*. Compilation is the same as when creating a clone for the target platform. After compiling and uploading the firmware to the microcontroller, the module is available via *import demo*. Exceeding the Flash memory range results in a compilation error.

directory ./ports/stm32/, compilation for Nucleo32 STM32L432KC

```
make BOARD=NUCLEO_L432KC clean
make BOARD=NUCLEO_L432KC
FROZEN_DIR=./my_modules
```

Microcontrollers with larger FLASH memory have a certain advantage, as a free part of the FLASH memory can be used as a local storage medium which is then mapped as a file system. The *os* library provides with elementary functions for creating and manipulating directories and files.

```
>>> import os
>>> help(os)
object <module 'uos'> is of type module
  __name__ -- uos
  uname -- <function>
  chdir -- <function>
  getcwd -- <function>
  ilistdir -- <function>
  listdir -- <function>
  mkdir -- <function>
  ...
  mount -- <function>
  umount -- <function>
```

The file system can be also used with external storage media, such as SD-Card. Support for communication with cards is a part of MicroPython. The file system is not only available in the MicroPython environment, but using an utility *./tools/pyboard.py* it is possible to save, delete files and export the directory structure outside the MicroPython environment.

```
usage: pyboard.py [-h] [--device DEVICE] [-b BAUDRATE]
                  [-u USER] [-p PASSWORD]
                  [-c COMMAND] [-w WAIT] [--follow] [-f]
                  [files [files ...]]
```

Additional arguments for -f (--filesystem)

```
ls
ls ./adresar
cp ./file ./
rm ./file
rmdir addr
mkdir addr
cat ./subor   file listing in the terminal
cat./subor > local.txt
```

Example

```
python pyboard.py -f ls
python pyboard.py -f cp ./test.py :
```

If the *main.py* file is overwritten or modified to contain executable code, it will automatically run after the next reset.

D. Implementation scalability

The elementary version of MicroPython without libraries is about 20KB in size after compilation. For FLASH memory usage optimization, creators of MicroPython added as many standard and supporting libraries as possible for every individual platform. Some implementation can thus differ in the manner of available libraries. Depending on the microcontroller's FLASH memory and RAM sizes, the firmware may contain the possibility of mapping part of the memory as a file system. This file system can also contain external storage media such e.g. SD/TF card and others. When some libraries and extension are not required, the firmware configuration may be modified and optimized according to current requirements. The *mpconfigboard.h* file is used to configure the target firmware for each individual platform. This file contains variables used to mark include or exclude tags for selected parts of the source code being prepared for the firmware compilation.

E. Implementation extensibility

While developing a new, non-standard device, the standard drivers included in MicroPython installation may not meet the requirements e.g. low-level peripheral operations and calculations may be needed. Likewise, with standard Python language there is also a possibility of extending MicroPython with native modules programmed in C/C++ language. Such modules can also use system libraries to control peripherals of the used microcontroller. Creating a module, the same way as in a standard Python, we need to program an interface between the native module in C/C++ and its representation in Python language. A thorough procedure of such procedure can be found in [13]. A number of program macros are used to create the interface, though with larger modules their structure can get quite large and complicated. Since the method of interfacing to native modules is widely standardized, there are available code generators for standard Python, e.g. SWIG, is used to generate the required interface based on the modules source code. In MicroPython is used the reverse procedure, we may use an interface generator (stub). Experimental implementations are shown in examples [11], [12], which create a stub consisting of program macros in C/C++ based on a function declaration in Python language.

III. Examples of application

A. Example 1 – Pin control

This example contains a simple module that consists of an initialization and a function that switches the pin with LED for a specified number of times and with an adjustable interval. The declaration of functions for interface generation has the following form:

```
# File blink.py, Python 3.7
# interface functions declarations
def init()-> None:
    """
    Function for port initialization and pin (with on-board
    LED)setup on NUCLEO_STM32L432KC is LED
    connected to PORTB, PIN3
    """
def blink(n: int, delay: int) -> None:
    """ LEDblinking
    :param n: number of blinks
    :param b: delay
    :return: None"""
```

This declaration is used to generate a stub. For stub generation is used [12] and the stub contains the implementation of the relevant function. We can use the procedure given in the manual to generate it, or we can use the following simple script.

```
# File gen_stub.py
# script for generating stub for micropython modules
following https://github.com/pazzarpj/micropython-ustubby

import ustubby
import sys
if len(sys.argv) < 1:
    print("Usage: python3.7 gen_stub.py input.py > output.c")
    exit()
md = sys.argv[1].split(".")[0]
stub = __import__(md)
s = ustubby.stub_module(stub)
print(s)
```

The implementation of the code itself can be created using HAL or LL. Example using low-level STM32 library is shown in the code snippet below (the full code is available on gihub [14]).

```
...
STATIC mp_obj_t blink_blink(mp_obj_t n_obj, mp_obj_t
delay_obj) {
    mp_int_t n = mp_obj_get_int(n_obj);
    mp_int_t delay = mp_obj_get_int(delay_obj);

    //Your code here
    // -----------------------------------------------------------------
    int i=0;
    for(i=0; i<n; i++){
        LL_mDelay(delay);
        LL_GPIO_TogglePin(GPIOB, LL_GPIO_PIN_3);
    }
    // -----------------------------------------------------------------

    return mp_const_none;
}
MP_DEFINE_CONST_FUN_OBJ_2(blink_blink_obj,
blink_blink);
...
```

Directory structure is then in the form shown in Fig. 3

```
my_project/
  |
  +-- modules/              <-- modules directory
  |  +--blink/
  |      +--blink.py        <-- source file for stub generator
  |       +--blink.c        <-- generated and modified source
code
  |      +--micropython.mk  <-- module configuration
  |
  +-- micropython/          <-- original MicroPython
     +--ports/              <-- ports for platforms
     ... +--stm32/        <-- directory for firmware compilation
     ...
```

Fig. 3 Directory structure for stub

Compilation using HAL libraries does not require any modifications. When compiling the implementation of modules with LL libraries, *./stm32/Makefile* must be modified as follows:

- add initialization of the USE_FULL_LL_DRIVER flag to make LL libraries available,
- add the compilation of LL libraries.

In the section CFLAGS add:

```
CFLAGS = $(INC) -Wall ...
...
```

```
CFLAGS += -DUSE_FULL_LL_DRIVER      <- added flag
```

In the section SRC_HAL add:

```
SRC_HAL = $(addprefix
$(HAL_DIR)/Src/stm32$(MCU_SERIES)xx_,\
   hal.c \
   ...
   hal_uart.c \
   ll_gpio.c \                <- added LL drivers no-prefix
   ...)
```

To compile the module, we still have to create a file *micropython.mk* in the module directory, which keeps the defined structure of the module, which itself can also consist of several files.

```
# File micropython.mk
BLINK_MOD_DIR := $(USERMOD_DIR)
SRC_USERMOD += $(BLINK_MOD_DIR)/blink.c
CFLAGS_USERMOD += -I$(BLINK_MOD_DIR)
```

Starting the compilation is then just alike in the previous case. In the directory *./ports/stm32/,* we start the compilation of the firmware:

```
make BOARD=NUCLEO_L432KC
USER_C_MODULES=../../../modules    CFLAGS_EXTRA=-
DMODULE_BLINK_ENABLED=1 all
```

After compiling we download the resulting file to the microcontroller using the standard procedure. Using the module in MicroPython is the same as using any other library:

```
>>> import blink
>>> blink.init()
>>> blink.blink(10, 100)
```

B. Example 2. DDS Generator

As an example of a larger module, we will show the implementation of DDS (Direct Digital Synthesis) generator. For several applications requiring an active excitation source, it is sometimes necessary to use a variable frequency harmonic signal source. For implementation, we will use the easy-to-implement DDS principle, which consists of a variable representing a 32-bit phase accumulator and a 32-bit phase step variable. The part of the accumulator word is used as the address of the value in the table of generated waveform values. For the synthesis of the harmonic waveform, we use the 8bit resolution of the DAC converter with a table length of 512 values. To transfer the values from the memory to the DAC, we will use DMA transfer. We will implement this in two steps using a double-buffer, while transferring the contents of one buffer to the DAC, we will prepare and initialize the contents of the other. The basic timer TIM6 is used to control the transmission, the output is PA4.

```
# Fil dds.py for stub generator
# Declaration of interface for DDS on STM32L4

def init(prescaler: int, period: int, )-> None:
   """
   Initialization of DMA, TIM6, DAC and PA4
   :param prescaler: prescaler setting for TIM6
   :param period: TIM6 period value - update step size
   :return: None
   """
def set(step: int) -> None:
   """
   Phase start
   :param step: phase increment
   :return: None
   """
```

The core of the DDS generator consists of a function that initializes the array with the values transferred in the DMA to the DAC converter at a rate defined by the timer. There are two arrays, while moving the first, the second one is filled in and the control for the MicroPython is released.

```
void DMA_Update(void){
   // callback function from DMA interrupt TC (Transfer
Complete)

   uint16_t index;

LL_DMA_DisableChannel(DMA1, LL_DMA_CHANNEL_
3);

   if(buffer_select == 0){     // buffer selection
      LL_DMA_SetMemoryAddress                    (DMA1,
      LL_DMA_CHANNEL_3, (uint32_t) &buffer_low); }

   else{

      LL_DMA_SetMemoryAddress                    (DMA1,
      LL_DMA_CHANNEL_3, (uint32_t) &buffer_high); }

   LL_DMA_SetDataLength(DMA1,
  LL_DMA_CHANNEL_3, BUFFER_SIZE);
   LL_DMA_EnableChannel(DMA1,
  LL_DMA_CHANNEL_3);       // DMA refresh

   for(index=0; index<BUFFER_SIZE; index++){
 // next buffer initialization
      phase_acc = phase_acc + phase_incr;
// DDS phase accumulator
      phase_addr = (phase_acc & 0x01FF0000) >> 16;
// mask for array of 512 table items – 9 bit
      if(buffer_select == 0){
         buffer_high[index]=     (SineTable[phase_addr]     *
ampl) >> 8;}

      else{

         buffer_low[index]     =     (SineTable[phase_addr]     *
ampl) >> 8;}
   }
   buffer_select = (buffer_select==0) ? 1 : 0;}

void DMA1_Channel3_IRQHandler(void){
   // Interrupt handler
   if( LL_DMA_IsActiveFlag_TC3(DMA1) !=0){
// Transfer complete
```

```
    DMA_Update();
    LL_DMA_ClearFlag_TC3(DMA1);
  }
  if(LL_DMA_IsActiveFlag_TE3(DMA1) == 1){
// Transfer error
}
}
```

Because the module size is larger, code itself is available on github [14]. We proceed with the compilation in the same way as in the previous example:

```
Make BOARD=NUCLEO_L476RG_EXP
USER_C_MODULES=../../../modules  CFLAGS_EXTRA=-
DMODULE_DDS_ENABLED
```

we come across one problem - the used IRQHandler is already reserved in MicroPython in the file *.ports/stm32/dma.c*, where it needs to be commented out. When initializing the module, we enter the parameters of the prescaler and the period of the counter, with the set method we set the phase increment (generator tuning):

```
>>> import dds
>>> dds.init(4,50)
>>> dds.set(0x00004000)
>>> dds.set(0x00028000)
```

It is obvious, that the resulting generator in Fig. 4 does not have the parameters of a laboratory device, but it is sufficient for common applications. With a clock frequency of the processor core of 80MHz (STM32L476RG), the generator can be used up to approx. 150kHz with jitter up to 1Hz.

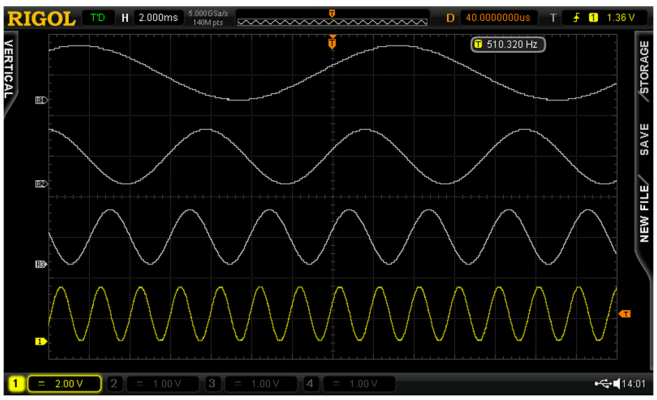

Fig. 4 DDS generator output

The frequency spectrum of the signal at the output in Fig. 5 without any output filter shows the distance of the second and third harmonics at the level of -20dB, the inclusion of the filter will of course significantly improve the properties of the generator.

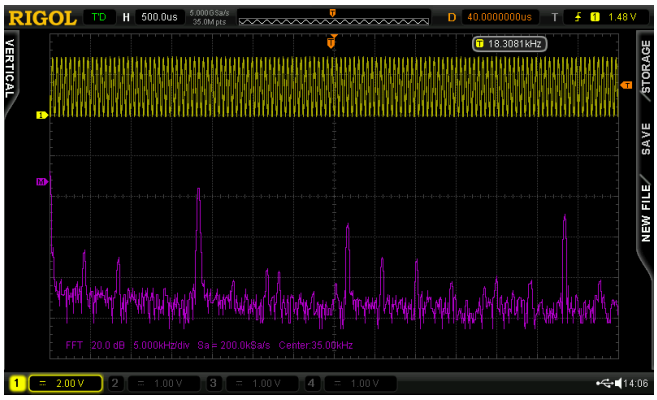

Fig. 5 DDS generator - frequency spectrum of the signal

IV. Conclusions

MicroPython on currently available microcontroller platforms provides effective option for the rapid development of IoT devices, in which the routine operation of standard high-level peripherals is implemented using verified and tested libraries. One of its advantages is possibility of focusing creative potential on the design and development of the application in hardware, driver, conversion and data processing without the need to solve general tasks bind with common operations. On the other hand, MicroPython allows creating modules using low-level programming, that can be compiled and used in the final firmware. With a suitable combination of its properties, it is possible to quickly implement tasks and assignments of practice, which under normal conditions can be time consuming. A distinctive advantage of such solution is portability. Developed firmware may be deployed not only within a single platform used for primary build, but also across a range of many solutions with implemented MicroPython. Such system properties could from multiple points of view represent a great added value in developing IoT devices intended for industrial usage, especially in the Industry 4.0 environment.

Acknowledgment

This work was supported in part by VEGA through the Holistic Approach of Knowledge Discovery From Production Data in Compliance With Industry 4.0 Concept Project under Grant 1/0272/18.

This paper was supported with project of basic research: Expanding the base of theoretical hypotheses and initiating assumptions to ensure scientific progress in methods of monitoring hydrometeors in the lower troposphere", which is funded by the R&D Incentives contract.

The authors acknowledge the support of the VEGA 2/0015/18 grant Meso- and micrometeorological exploration of the occurrence of hydrometeors in boundary layer of troposphere based on passive evaluation of changes of electromagnetic radiation from anthropogenic sources.

References

[1] MATUSOV, Jozef, Marian MOKRY, Zuzana KOLKOVA a Stefan SEDIVY. Intelligent systems installed in building of research centre for research purposes. 2016, , 020033-. DOI: 10.1063/1.4953727.

[2] SEDIVY, Stefan, Lenka MICECHOVA, Pavel SCHEBER. Mechatronics Solutions in Process of Transport Infrastructure

Monitoring and Diagnostics. Mechatronics 2019: Recent Advances Towards Industry 4.0. Cham: Springer International Publishing, 2020, 2020-08-28, , 141-148. Advances in Intelligent Systems and Computing. DOI: 10.1007/978-3-030-29993-4_18. ISBN 978-3-030-29992-7.

[3] GASPAR, Gabriel, Peter FABO, Michal KUBA, Juraj DUDAK a Eduard NEMLAHA. MicroPython as a Development Platform for IoT Applications. Intelligent Algorithms in Software Engineering. Cham: Springer International Publishing, 2020, 2020-08-09, , 388-394. Advances in Intelligent Systems and Computing. ISBN 978-3-030-51964-3.

[4] ZIKRIA, Yousaf Bin, Sung Won KIM, Oliver HAHM, Muhammad Khalil AFZAL a Mohammed Y. AALSALEM. Internet of Things (IoT) Operating Systems Management: Opportunities, Challenges, and Solution. Sensors. 2019, 19(8). DOI: 10.3390/s19081793. ISSN 1424-8220. Available at: https://www.mdpi.com/1424-8220/19/8/1793

[5] Zephyr Project | Home [online]. San Francisco: Linux Foundation, 2020 [cit. 2020-06-18]. Available at: https://www.zephyrproject.org/

[6] MicroPython - Python for microcontrollers [online]. Cambridge: George Robotics, 2020 [cit. 2020-06-18]. Available at: http://micropython.org/

[7] Welcome to FPGA MicroPython (FμPy) [online]. San Francisco: GitHub, 2020 [cit. 2020-06-18]. Available at: https://fupy.github.io/

[8] Micropython/ports/zephyr at master · micropython/micropython · GitHub [online]. San Francisco: GitHub, 2020 [cit. 2020-06-18]. Available at: https://github.com/micropython/micropython/tree/master/ports/zephyr

[9] 12-Bit + Temp Sensor & Thermal Window Comparator w/2-Wire Interface datasheet (Rev. D) - lm92.pdf [online]. Dallas: Texas Instruments, 2020 [cit. 2020-06-18]. Available at: https://www.ti.com/lit/ds/symlink/lm92.pdf

[10] Running Python in the Linux Kernel - Yonatan Goldschmidt - Medium [online]. San Francisco: Medium, 2020 [cit. 2020-06-18]. Available at: https://medium.com/@yon.goldschmidt/running-python-in-the-linux-kernel-7cbcbd44503c

[11] GitHub - pazzarpj/micropython-ustubby: Library for building micropython c extensions using python [online]. San Francisco: GitHub, 2020 [cit. 2020-06-18]. Available at: https://github.com/pazzarpj/micropython-ustubby

[12] GitHub - SummerLife/micropython-ustubby: Library for building micropython c extensions using python [online]. San Francisco: GitHub, 2020 [cit. 2020-06-18]. Available at: https://github.com/SummerLife/micropython-ustubby

[13] MicroPython external C modules — Pycopy 3.1.5 documentation [online]. Oregon: Read the Docs, 2020 [cit. 2020-06-18]. Available at: https://pycopy.readthedocs.io/en/latest/develop/cmodules.html#cmodules

[14] GitHub - pfabo/STM32-MicroPython-Modules: Examples for creating user-defined modules in python [online]. San Francisco: GitHub, 2020 [cit. 2020-06-18]. Available at: https://github.com/pfabo/STM32-MicroPython-Modules